INFORME DE LABORATORIO

INFORMACIÓN BÁSICA					
ASIGNATURA:	Sistemas Distribuidos				
TÍTULO DE LA PRÁCTICA:	Algoritmos de Sincronización				
NÚMERO DE LA PRÁCTICA:	02	**AÑO LECTIVO:**	2026	**NRO. SEMESTRE:**	2026-A
FECHA DE PRESENTACIÓN:	28/04/2026	**HORA DE PRESENTACIÓN:**	11:59:00		
INTEGRANTE(s): - Bedregal Perez Daniel - Jara Mamani Mariel Alisson - Mestas Zegarra Christian Raul - Noa Camino Yenaro Joel - Sequeiros Condori Luis Gustavo				**NOTA (0 - 20):**	Nota colocada por el docente
DOCENTE: Mg. Maribel Molina Barriga					

RESULTADOS Y PRUEBAS

ENLACE A GITHUB

https://github.com/christianmz565/sd-2026-lab-c-grupo-3/tree/main/l2

SOLUCIÓN DE EJERCICIOS RESUELTOS

Lamport Clock

El estado compartido del reloj se concentra en `tick()`, `update(...)` y `getTime()` porque el algoritmo necesita que cada evento local aumente una marca monotónica y que cada recepción combine los datos remotos con los locales mediante `max(local, recibido) + 1`.

```java
private int clock;

public LamportClock() {
  this.clock = 0;
}

public synchronized int tick() {
  this.clock++;
  return this.clock;
}

public synchronized void update(int receivedTime) {
  this.clock = Math.max(this.clock, receivedTime) + 1;
}
```

El cierre de ejecución usa `join()` sobre cada hilo para poder imprimir el sistema completo, ya que sin esa barrera el proceso principal podría terminar antes de que algunos eventos actualicen el reloj y el valor reportado podría ser incorrecto.

```java
for (Thread thread : threads) {
  try {
    thread.join();
  } catch (InterruptedException e) {
    e.printStackTrace();
  }
```

```
    }
    System.out.println("Final Lamport time: " + clock.getTime());
```

A continuación, se muestra el resultado de compilar y ejecutar el programa:

```
❯ javac l2/src/s1/LamportClock.java
Note: l2/src/s1/LamportClock.java uses or overrides a deprecated API.
Note: Recompile with -Xlint:deprecation for details.
❯ java -cp l2/src/s1 LamportClock
Thread 29 created event with Lamport time 2
Thread 31 created event with Lamport time 4
Thread 32 created event with Lamport time 5
Thread 28 created event with Lamport time 1
Thread 30 created event with Lamport time 3
Thread 29 received event with Lamport time 6
Thread 28 received event with Lamport time 8
Thread 32 received event with Lamport time 10
Thread 30 received event with Lamport time 12
Thread 31 received event with Lamport time 14
Final Lamport time: 15
```

SOLUCIÓN DE EJERCICIOS PROPUESTOS

Cristian Algorithm

El diseño cliente-servidor separa la autoridad temporal en `TimeServer` y la corrección en `ClientNode` para modelar el objetivo del algoritmo de Cristian, que no es compartir estado global entre nodos sino permitir que cada cliente ajuste su desviación respecto a una referencia común.

```java
static class TimeServer {

  private final long serverOffsetMillis;

  TimeServer(long serverOffsetMillis) {
    this.serverOffsetMillis = serverOffsetMillis;
  }

  synchronized long getCurrentTimeMillis() {
    return System.currentTimeMillis() + serverOffsetMillis;
  }
}

static class ClientNode implements Runnable {

  private final String name;
  private final TimeServer server;
  private final Random random;
  private long localOffsetMillis;

  ClientNode(String name, TimeServer server, long initialOffsetMillis) {
    this.name = name;
    this.server = server;
    this.localOffsetMillis = initialOffsetMillis;
    this.random = new Random();
  }

  synchronized long localTimeMillis() {
    return System.currentTimeMillis() + localOffsetMillis;
  }

  synchronized void adjustClock(long adjustmentMillis) {
    localOffsetMillis += adjustmentMillis;
  }
```

La fase de sincronización mide `requestTime` y `responseTime` para estimar `RTT` y calcula `adjustment = (serverTime + RTT/2) - localTime` porque decide compensar el costo de red antes de corregir el reloj y así reducir el sesgo externo.

```java
void synchronizeClock() {
  long beforeSync = localTimeMillis();

  long requestTime = System.currentTimeMillis();
  simulateNetworkDelay();
  long serverTime = server.getCurrentTimeMillis();
  simulateNetworkDelay();
  long responseTime = System.currentTimeMillis();

  long roundTripTime = responseTime - requestTime;
  long estimatedServerTime = serverTime + (roundTripTime / 2);
  long adjustment = estimatedServerTime - localTimeMillis();

  adjustClock(adjustment);
  long afterSync = localTimeMillis();

  System.out.println(
    name +
      " | Antes: " +
      beforeSync +
      " | RTT: " +
      roundTripTime +
      " ms" +
      " | Ajuste: " +
      adjustment +
      " ms" +
      " | Despues: " +
      afterSync
  );
}
```

El `main` crea clientes con offsets positivos y negativos y los ejecuta para provocar condiciones de desfase.

```java
for (ClientNode client : clients) {
  Thread thread = new Thread(client);
  threads.add(thread);
  thread.start();
}

for (Thread thread : threads) {
  try {
    thread.join();
  } catch (InterruptedException e) {
    Thread.currentThread().interrupt();
  }
}
```

A continuación, se muestra el resultado de compilar y ejecutar el programa:

```
❯ javac l2/src/e1/CristianAlgorithm.java
❯ java -cp l2/src/e1 CristianAlgorithm
Cliente-4 | Antes: 1776806744565 | RTT: 198 ms | Ajuste: 1531 ms | Despues: 1776806746294
Cliente-3 | Antes: 1776806748065 | RTT: 256 ms | Ajuste: -2046 ms | Despues: 1776806746275
Cliente-1 | Antes: 1776806747264 | RTT: 264 ms | Ajuste: -1253 ms | Despues: 1776806746275
Cliente-2 | Antes: 1776806745164 | RTT: 273 ms | Ajuste: 912 ms | Despues: 1776806746349
Sincronizacion de Cristian completada.
```

Berkeley Algorithm

El modelo divide `Node` y `Coordinator` ya que Berkeley asume que la sincronización puede resolverse por coordinación interna, donde cada nodo expone su desfase local y el coordinador centraliza la decisión de ajuste sin requerir una fuente externa de tiempo absoluto.

```java
static class Node {

  private final String name;
  private long offsetMillis;
```

```java
    Node(String name, long offsetMillis) {
      this.name = name;
      this.offsetMillis = offsetMillis;
    }

    synchronized long getLocalTimeMillis() {
      return System.currentTimeMillis() + offsetMillis;
    }

    synchronized long getOffsetMillis() {
      return offsetMillis;
    }

    synchronized void adjustOffset(long adjustmentMillis) {
      offsetMillis += adjustmentMillis;
    }

    String getName() {
      return name;
    }
  }

  static class Coordinator {

    private final Node master;
    private final List<Node> nodes;
    private final Random random;

    Coordinator(Node master, List<Node> nodes) {
      this.master = master;
      this.nodes = nodes;
      this.random = new Random();
    }
```

En la ronda de cálculo el coordinador compara cada reloj con el maestro, promedia las diferencias y asigna a cada nodo un ajuste `averageDifference - nodeDiff` porque el objetivo no es copiar un reloj único sino minimizar la dispersión.

```java
      for (Node node : nodes) {
        simulateNetworkDelay();
        long nodeTime = node.getLocalTimeMillis();
        long difference = nodeTime - masterTime;
        observedOffsets.add(difference);
      }

      long total = 0;
      for (long diff : observedOffsets) {
        total += diff;
      }
      long averageDifference = total / observedOffsets.size();

      for (int i = 0; i < nodes.size(); i++) {
        Node node = nodes.get(i);
        long nodeDiff = observedOffsets.get(i);
        long adjustment = averageDifference - nodeDiff;
        node.adjustOffset(adjustment);
        System.out.println("Ajuste para " + node.getName() + ": " + adjustment + " ms");
      }
```

En `main` se fija un conjunto con desfases amplios y se ejecuta una sola ronda que logra reducir el error relativo entre nodos antes de usar políticas adicionales como exclusión de extremos o rondas repetidas.

```java
  public static void main(String[] args) {
    Node master = new Node("Master", 500);
    List<Node> nodes = new ArrayList<>();

    nodes.add(master);
    nodes.add(new Node("Nodo-1", -1800));
    nodes.add(new Node("Nodo-2", 2200));
    nodes.add(new Node("Nodo-3", 900));
```

```
    nodes.add(new Node("Nodo-4", -600));

    Coordinator coordinator = new Coordinator(master, nodes);
    coordinator.synchronize();
  }
```

A continuación, se muestra el resultado de compilar y ejecutar el programa:

```
❯ javac l2/src/e2/BerkeleyAlgorithm.java
❯ java -cp l2/src/e2 BerkeleyAlgorithm
Estado inicial:
Master -> offset=500 ms, localTime=1776806754549
Nodo-1 -> offset=-1800 ms, localTime=1776806752296
Nodo-2 -> offset=2200 ms, localTime=1776806756296
Nodo-3 -> offset=900 ms, localTime=1776806754996
Nodo-4 -> offset=-600 ms, localTime=1776806753496
Ajuste para Master: -51 ms
Ajuste para Nodo-1: 2152 ms
Ajuste para Nodo-2: -1977 ms
Ajuste para Nodo-3: -776 ms
Ajuste para Nodo-4: 651 ms

Estado final:
Master -> offset=449 ms, localTime=1776806755002
Nodo-1 -> offset=352 ms, localTime=1776806754905
Nodo-2 -> offset=223 ms, localTime=1776806754776
Nodo-3 -> offset=124 ms, localTime=1776806754677
Nodo-4 -> offset=51 ms, localTime=1776806754604
```

CUESTIONARIO

¿Por qué es conveniente el uso de relojes lógicos en lugar de relojes físicos?
Los relojes lógicos son convenientes en sistemas distribuidos porque el problema principal en muchos protocolos no es conocer la hora absoluta sino preservar el orden causal entre eventos relacionados por comunicación. Un reloj físico puede contener deriva, desfase por latencia y errores de sincronización externa, mientras que un reloj lógico evita esas dependencias y permite razonar sobre consistencia de ejecución con reglas deterministas de incremento y actualización

Explique ¿Cuál algoritmo entre Cristian y Berkeley resuelve mejor la sincronización?
Cristian es más adecuado cuando existe un servidor de tiempo confiable y de baja latencia porque cada cliente realiza ajuste directo y rápido con una fórmula simple basada en RTT, mientras que Berkeley es más adecuado cuando se prioriza autonomía de cluster y no se dispone de fuente externa estricta. En esta práctica Cristian mostró convergencia puntual por cliente hacia la referencia del servidor, y Berkeley mostró convergencia grupal por promedio con reducción de dispersión global, por lo cual la elección correcta es contextual según topología, tolerancia a falla y objetivo operativo

Dado tres procesos P1, P2 y P3, representa la planificación de los procesos siguiendo el algoritmo de "ocurre antes de".
Se propone la siguiente planificación con tres procesos y dos mensajes para ilustrar la relación ocurre antes entre eventos locales y remotos

Proceso	Evento	Tiempo Lamport	Relación ocurre antes
P1	a: evento local	1	Inicio de secuencia en P1
P1	b: envia m1 a P2	2	a -> b por orden local
P2	c: recibe m1	3	b -> c por regla de mensaje
P2	d: envia m2 a P3	4	c -> d por orden local
P3	e: recibe m2	5	d -> e por regla de mensaje
P3	f: evento local	6	e -> f por orden local

La cadena resultante: `a -> b -> c -> d -> e -> f` representa una historia causal consistente y cualquier evento no conectado por estas relaciones se considera concurrente en el marco de Lamport

CONCLUSIONES Y RECOMENDACIONES

CONCLUSIONES

1. La ejecución de `LamportClock` verificó que la marca temporal lógica mantiene orden causal entre eventos concurrentes mediante incremento local y ajuste por máximo recibido
2. La implementación de `CristianAlgorithm` mostró convergencia de relojes cliente hacia una referencia de servidor con compensación por retardo de ida y vuelta
3. La implementación de `BerkeleyAlgorithm` mostró reducción de dispersión temporal del conjunto al aplicar ajustes calculados desde la media de desfases observados

RECOMENDACIONES

1. Ejecutar varias rondas por algoritmo con semillas de retardo controladas para cuantificar error medio y varianza de convergencia en diferentes condiciones de red
2. Incorporar umbrales para excluir outliers temporales en Berkeley y evitar que nodos con deriva extrema distorsionen el promedio global
3. Extender los ejemplos con fallas de nodo y pérdida de mensajes para evaluar robustez de sincronización ante condiciones no ideales de sistema distribuido

REFERENCIAS Y BIBLIOGRAFÍA

[1] Tanenbaum, A.S. (2008). Sistemas distribuidos: principios y paradigmas. México. Pearson Educación.

[2] Ceballos, F. J. (2006). Java 2, Curso de programación. México: Alfaomega, RaMa.

[3] Deitel, H. M., & Deitel, P. J. (2004). Cómo programar en Java. México: Pearson Educación.

[4] García Tomás, J., Ferrando, S., & Piattini, M. (2001). Redes para procesos distribuidos. México: Alfaomega Ra-Ma.

[5] Orfali, R., & Harkey, D. (1998). Client/Server Programming with Java and CORBA. USA: Wiley.

ANEXOS

Código completo: Lamport Clock

```java
import java.util.ArrayList;
import java.util.List;

public class LamportClock {

  private int clock;

  public LamportClock() {
    this.clock = 0;
  }

  public synchronized int tick() {
    this.clock++;
    return this.clock;
  }

  public synchronized void update(int receivedTime) {
    this.clock = Math.max(this.clock, receivedTime) + 1;
  }
```

```java
  public int getTime() {
    return this.clock;
  }

  public static void main(String[] args) {
    List<Thread> threads = new ArrayList<>();
    LamportClock clock = new LamportClock();
    for (int i = 0; i < 5; i++) {
      Thread thread = new Thread(
        new Runnable() {
          @Override
          public void run() {
            int time = clock.tick();

            System.out.println(
              "Thread " +
                Thread.currentThread().threadId() +
                " created event with Lamport time " +
                time
            );
            try {
              Thread.sleep((long) (Math.random() * 1000));
            } catch (InterruptedException e) {
              e.printStackTrace();
            }

            int receivedTime = clock.tick();
            System.out.println(
              "Thread " +
                Thread.currentThread().threadId() +
                " received event with Lamport time " +
                receivedTime
            );
            clock.update(receivedTime);
          }
        }
      );
      threads.add(thread);
      thread.start();
    }
    for (Thread thread : threads) {
      try {
        thread.join();
      } catch (InterruptedException e) {
        e.printStackTrace();
      }
    }
    System.out.println("Final Lamport time: " + clock.getTime());
  }
}
```

Código completo: Cristian Algorithm

```java
import java.util.ArrayList;
import java.util.List;
import java.util.Random;

class CristianAlgorithm {

  static class TimeServer {

    private final long serverOffsetMillis;

    TimeServer(long serverOffsetMillis) {
      this.serverOffsetMillis = serverOffsetMillis;
    }

    synchronized long getCurrentTimeMillis() {
      return System.currentTimeMillis() + serverOffsetMillis;
    }
  }
```

```java
static class ClientNode implements Runnable {

  private final String name;
  private final TimeServer server;
  private final Random random;
  private long localOffsetMillis;

  ClientNode(String name, TimeServer server, long initialOffsetMillis) {
    this.name = name;
    this.server = server;
    this.localOffsetMillis = initialOffsetMillis;
    this.random = new Random();
  }

  synchronized long localTimeMillis() {
    return System.currentTimeMillis() + localOffsetMillis;
  }

  synchronized void adjustClock(long adjustmentMillis) {
    localOffsetMillis += adjustmentMillis;
  }

  private void simulateNetworkDelay() {
    try {
      Thread.sleep(50 + random.nextInt(150));
    } catch (InterruptedException e) {
      Thread.currentThread().interrupt();
    }
  }

  void synchronizeClock() {
    long beforeSync = localTimeMillis();

    long requestTime = System.currentTimeMillis();
    simulateNetworkDelay();
    long serverTime = server.getCurrentTimeMillis();
    simulateNetworkDelay();
    long responseTime = System.currentTimeMillis();

    long roundTripTime = responseTime - requestTime;
    long estimatedServerTime = serverTime + (roundTripTime / 2);
    long adjustment = estimatedServerTime - localTimeMillis();

    adjustClock(adjustment);
    long afterSync = localTimeMillis();

    System.out.println(
      name +
        " | Antes: " +
        beforeSync +
        " | RTT: " +
        roundTripTime +
        " ms" +
        " | Ajuste: " +
        adjustment +
        " ms" +
        " | Despues: " +
        afterSync
    );
  }

  @Override
  public void run() {
    synchronizeClock();
  }
}

public static void main(String[] args) {
  TimeServer server = new TimeServer(0);
  List<Thread> threads = new ArrayList<>();
```

```
    ClientNode[] clients = new ClientNode[] {
      new ClientNode("Cliente-1", server, 1200),
      new ClientNode("Cliente-2", server, -900),
      new ClientNode("Cliente-3", server, 2000),
      new ClientNode("Cliente-4", server, -1500),
    };

    for (ClientNode client : clients) {
      Thread thread = new Thread(client);
      threads.add(thread);
      thread.start();
    }

    for (Thread thread : threads) {
      try {
        thread.join();
      } catch (InterruptedException e) {
        Thread.currentThread().interrupt();
      }
    }

    System.out.println("Sincronizacion de Cristian completada.");
  }
}
```

Código completo: Berkeley Algorithm

```
import java.util.ArrayList;
import java.util.List;
import java.util.Random;

class BerkeleyAlgorithm {

  static class Node {

    private final String name;
    private long offsetMillis;

    Node(String name, long offsetMillis) {
      this.name = name;
      this.offsetMillis = offsetMillis;
    }

    synchronized long getLocalTimeMillis() {
      return System.currentTimeMillis() + offsetMillis;
    }

    synchronized long getOffsetMillis() {
      return offsetMillis;
    }

    synchronized void adjustOffset(long adjustmentMillis) {
      offsetMillis += adjustmentMillis;
    }

    String getName() {
      return name;
    }
  }

  static class Coordinator {

    private final Node master;
    private final List<Node> nodes;
    private final Random random;

    Coordinator(Node master, List<Node> nodes) {
      this.master = master;
      this.nodes = nodes;
      this.random = new Random();
    }
```

```java
    private void simulateNetworkDelay() {
      try {
        Thread.sleep(40 + random.nextInt(110));
      } catch (InterruptedException e) {
        Thread.currentThread().interrupt();
      }
    }

    void synchronize() {
      List<Long> observedOffsets = new ArrayList<>();
      long masterTime = master.getLocalTimeMillis();

      System.out.println("Estado inicial:");
      for (Node node : nodes) {
        System.out.println(
          node.getName() +
            " -> offset=" +
            node.getOffsetMillis() +
            " ms, localTime=" +
            node.getLocalTimeMillis()
        );
      }

      for (Node node : nodes) {
        simulateNetworkDelay();
        long nodeTime = node.getLocalTimeMillis();
        long difference = nodeTime - masterTime;
        observedOffsets.add(difference);
      }

      long total = 0;
      for (long diff : observedOffsets) {
        total += diff;
      }
      long averageDifference = total / observedOffsets.size();

      for (int i = 0; i < nodes.size(); i++) {
        Node node = nodes.get(i);
        long nodeDiff = observedOffsets.get(i);
        long adjustment = averageDifference - nodeDiff;
        node.adjustOffset(adjustment);
        System.out.println("Ajuste para " + node.getName() + ": " + adjustment + " ms");
      }

      System.out.println("\nEstado final:");
      for (Node node : nodes) {
        System.out.println(
          node.getName() +
            " -> offset=" +
            node.getOffsetMillis() +
            " ms, localTime=" +
            node.getLocalTimeMillis()
        );
      }
    }
  }

  public static void main(String[] args) {
    Node master = new Node("Master", 500);
    List<Node> nodes = new ArrayList<>();

    nodes.add(master);
    nodes.add(new Node("Nodo-1", -1800));
    nodes.add(new Node("Nodo-2", 2200));
    nodes.add(new Node("Nodo-3", 900));
    nodes.add(new Node("Nodo-4", -600));

    Coordinator coordinator = new Coordinator(master, nodes);
    coordinator.synchronize();
  }
}
```